

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики-процессов управления

Программа бакалавриата

“Большие данные и распределенная цифровая платформа”

ОТЧЕТ

по лабораторной работе №5

по дисциплине «Алгоритмы и структуры данных»

**на тему «Разработка и реализация алгоритма роевого интеллекта для
решения задач глобальной оптимизации»**

Вариант – 1

**Студент гр. 23Б15-пу
Трофимов И. А.**

**Преподаватель
Дик А.Г.**

Санкт-Петербург

2024 г.

Оглавление

1. Цель работы	3
2. Теоретическая часть	3
3. Описание задачи	6
4. Основные шаги программы	7
5. Блок схема	7
6. Описание программы	8
7. Рекомендации пользователя	10
8. Рекомендации программиста	10
9. Исходный код программы:	11
10. Контрольный пример	11
11. Исследование	12
12. Вывод	15
13. Источники	15

Цель работы

Целью данной работы является исследование и разработка алгоритма роевого интеллекта для решения задач глобальной оптимизации, а также его сравнительный анализ с генетическим алгоритмом. В ходе выполнения работы предполагается изучить особенности алгоритмов роевого интеллекта, реализовать программу для поиска минимума выбранной тестовой функции, протестировать её эффективность и оценить качество решения.

Теоретическая часть

Алгоритм роевого интеллекта (PSO):

Алгоритм роевого интеллекта (PSO, Particle Swarm Optimization) был предложен Джеймсом Кеннеди и Ричардом Эберхартом в 1995 году как метод оптимизации, основанный на наблюдениях за поведением социальных животных, таких как стаи птиц или рои муравьёв. Алгоритм моделирует коллективное поведение группы частиц, которые в процессе поиска решения взаимодействуют друг с другом и обновляют свои позиции в поисковом пространстве на основе личных и глобальных достижений.

Основные компоненты алгоритма:

1. **Частица:** Каждая частица в алгоритме представляет собой возможное решение задачи. Частицы могут быть представлением точек в многомерном пространстве поиска. Каждая частица имеет:
 - Позицию x_i , которая представляет собой решение.
 - Скорость v_i , которая определяет, как быстро частица движется в пространстве.
 - Личное наилучшее решение p_i , которое соответствует наименьшему значению функции цели, найденному данной частицей.

- Глобальное наилучшее решение g^* , которое является наименьшим значением функции цели среди всех частиц.

2. Обновление позиции и скорости: Каждая частица обновляет свою позицию на каждом шаге с использованием следующей формулы:

$$\begin{aligned} \mathbf{v}_i(t+1) &= w \cdot \mathbf{v}_i(t) + c_1 \cdot r_1 \cdot (\mathbf{p}_i - \mathbf{x}_i(t)) + c_2 \cdot r_2 \cdot (\mathbf{g}^* - \mathbf{x}_i(t)) \\ \mathbf{x}_i(t+1) &= \mathbf{x}_i(t) + \mathbf{v}_i(t+1) \end{aligned}$$

Рис.1 Формулы обновления позиции и скорости

где:

- $\mathbf{v}_i(t)$ — скорость частицы на текущем шаге.
- $\mathbf{x}_i(t)$ — позиция частицы на текущем шаге.
- w — коэффициент инерции, который управляет тем, насколько сильно предыдущая скорость влияет на текущую.
- c_1, c_2 — коэффициенты, которые регулируют влияние личного и глобального лучшего решения на обновление скорости.
- r_1, r_2 — случайные значения, генерируемые в пределах от 0 до 1.
- $\mathbf{p}_i$ — личное наилучшее решение частицы.
- $\mathbf{g}^*$ — глобальное наилучшее решение среди всех частиц.

Обновление скорости и позиции позволяет частицам двигаться по поисковому пространству, постепенно приближаясь к оптимальному решению.

Модификация алгоритма: Ограничение скорости

Ограничение скорости — это механизм, который используется для предотвращения чрезмерных изменений в положении частиц в поисковом пространстве, что может повлиять на эффективность поиска оптимального решения.

Влияние ограничения скорости на сходимость

1. Предотвращение скачков и нестабильности:

- Ограничение скорости помогает предотвратить слишком большие изменения в положении частиц, что может вызвать «скачки» между решениями, особенно в случае сильных случайных компонентов в обновлениях скорости.
- Это может быть полезно в случаях, когда частицы слишком быстро покидают область, где находятся оптимальные решения, из-за слишком высоких значений скорости.

2. Ускорение сходимости в стабильных областях:

- Когда скорость ограничена, частицы не могут перемещаться слишком быстро, что дает им больше времени для исследования окружающей среды и нахождения локальных или глобальных минимумов.
- Это может ускорить сходимость в стабильных областях, где частицы должны постепенно сходиться к оптимальному решению.

3. Уменьшение вероятности застревания в локальных минимумах:

- Ограничение скорости снижает риск того, что частицы будут слишком агрессивно двигаться в поиске оптимума и застрять в локальных минимумах. Это особенно важно, если функция имеет много локальных экстремумов.
- Частицы не будут делать большие «скачки» и могут продолжать исследовать более мелкие участки пространства, избегая быстрых, случайных перемещений, которые могут привести к неправильным решениям.

4. Риск замедления сходимости:

- С другой стороны, если скорость слишком сильно ограничена, частицы могут двигаться очень медленно, что приведет к очень

долгому поиску оптимума. Это может замедлить сходимость в случае, если частицы находятся вблизи глобального оптимума, но не могут быстро его достичь.

- Это может быть особенно актуально, если поисковое пространство большое или сложное.

5. Влияние на адаптивную скорость (если она используется):

- Если используется адаптивная скорость (например, уменьшение инерции с каждой итерацией), ограничение скорости может работать в тандеме с адаптивной скоростью, чтобы сбалансировать исследование пространства и сходимость к решению. В начальной стадии исследования может быть полезно большее изменение положения частиц, а на поздних этапах — медленное приближение к оптимуму.

Таким образом, использование ограничения скорости помогает улучшить эффективность алгоритма, особенно на этапах, когда частицы находятся близко к оптимальному решению и требуется точность.

Описание задачи

В рамках данной лабораторной работы необходимо исследовать эффективность роевого алгоритма при решении задачи оптимизации, а именно — при минимизации функции: $f(x,y)=x^2+3y^2+2xy$.

Алгоритм роевого интеллекта (PSO):

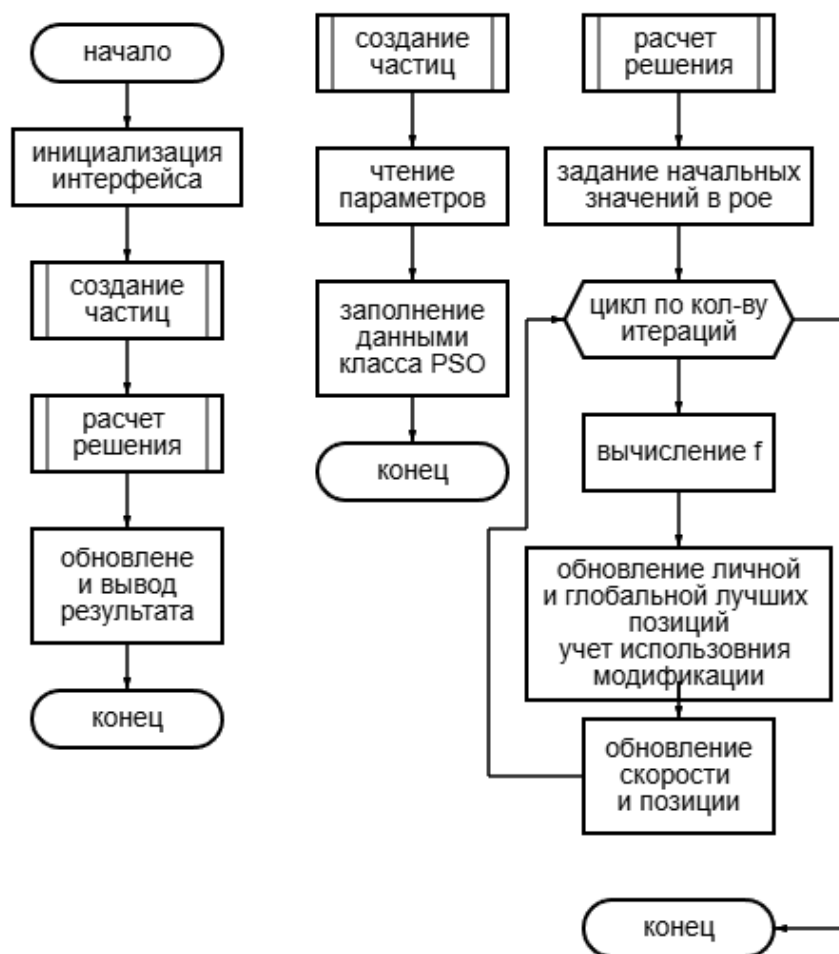
- Алгоритм будет работать, используя несколько частиц, каждая из которых будет представлять собой пару значений.
- Частицы будут обновлять свою позицию и скорость на основе своих собственных лучших решений и глобальных лучших решений среди всех частиц.

- Целью является минимизация функции, то есть нахождение таких значений x и y , которые дают минимальное значение функции.

Основные шаги:

1. Инициализация положения и скорости частиц в пространстве поиска.
2. Оценка текущей позиции каждой частицы с использованием целевой функции.
3. Обновление скорости и позиции каждой частицы в зависимости от их индивидуальной истории и глобального лучшего решения.
4. Повторение шагов 2 и 3 до достижения заданного числа итераций или удовлетворительного результата (приближение к минимальному значению функции).

Блок схема программы



Описание программы

Программная реализация написана на языке Python версии 3.13.0 с использованием библиотек `random`[\[1\]](#), `tkinter`[\[2\]](#). Программа представляет собой GUI-приложение, реализующее алгоритм PSO для поиска минимального значения целевой функции. Пользователь может настраивать параметры алгоритма, такие как количество частиц и коэффициенты инерции. Кроме того, предусмотрена возможность включения модификации алгоритма, что позволяет изменять динамику обновления частиц. В процессе разработки программы использовался следующий модуль:

Таблица1 main.py

Функция	Описание	Возвращаемое значение
<code>reset()</code>	Сбрасывает позицию и скорость частицы к их начальному состоянию, а также обновляет лучшие найденные значения	None
<code>target_function()</code>	Вычисляет значение функции	float
<code>update_best()</code>	Обновляет лучшие найденные значения для частицы, если текущее значение целевой функции для текущей позиции меньше, чем ранее найденное лучшее значение	None

update_velocity()	Обновляет скорость частицы с использованием формулы	None
move()	Перемещает частицу на новое положение	None
reset_particles()	Функция сбрасывает все частицы в их начальное состояние, включая глобальное лучшее положение и значение	None
update_particles()	Обновляет скорость и позицию каждой частицы в системе, а также проверяет, нужно ли обновить глобальное лучшее положение и значение	None
run()	Выполняет весь процесс оптимизации, обновляя частицы в течение заданного количества итераций	None
create_interface()	Создает графический интерфейс	None

<code>generate_particles()</code>	Создает экземпляр алгоритма PSO с параметрами	None
<code>calculate_solution()</code>	Сбрасывает частицы, выполняет оптимизацию с заданным количеством итераций и выводит результат	None
<code>plot_particles()</code>	Рисует частицы на холсте	None

Рекомендации пользователя

Для запуска программы убедитесь, что у вас установлен Python и необходимые библиотеки, такие как tkinter[\[2\]](#). Код можно запустить в среде разработки или через командную строку, используя консоль для настройки параметров и генерации данных. Запуск программы производится через файл `main.py`.

При запуске программы вам будет предложено выбрать параметры запускаемого алгоритма. Вводите ответы соответствующие поля в GUI. Результаты работы будут представлены в таблице.

Рекомендации программиста

Для поддержания актуальности и работоспособности программы используйте последние версии библиотек, особенно tkinter[\[2\]](#). Применяйте практики

надлежащего именования переменных и функций для улучшения читаемости кода.

Исходный код программы:

[Labs/5 at commt-5 · hanglider/Labs](#)

Контрольный пример

1. Запуск программы

Для запуска программы используйте файл `main.py`. Программа загружает GUI. (Рис. 1)

```
C:\IT\Labs\Labs\5>python main.py
```

Рис 1. Результат работы алгоритма

2. Выбор параметров PSO

После запуска программы пользователю будет предложено выбрать параметры (Рис. 2)

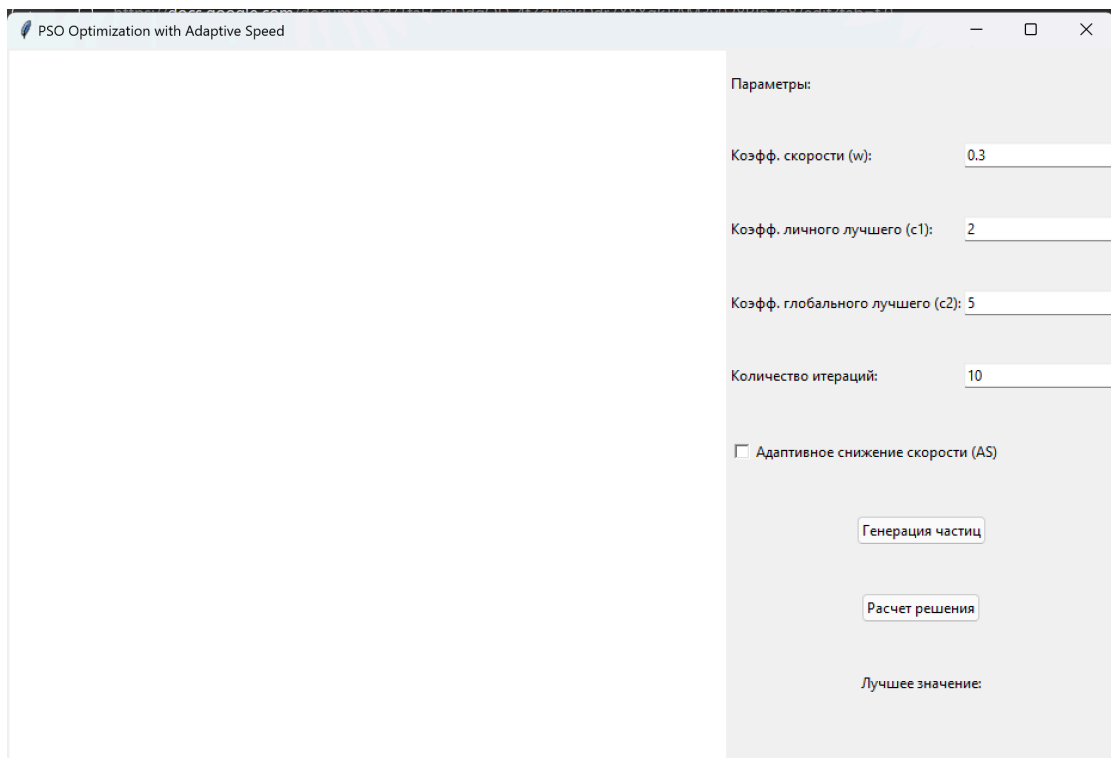


Рис 2. Результат работы алгоритма

3. Обработка данных и вывод результатов

После выбора параметров программа запускает алгоритм, а затем выводит результаты на графике (Рис. 3).

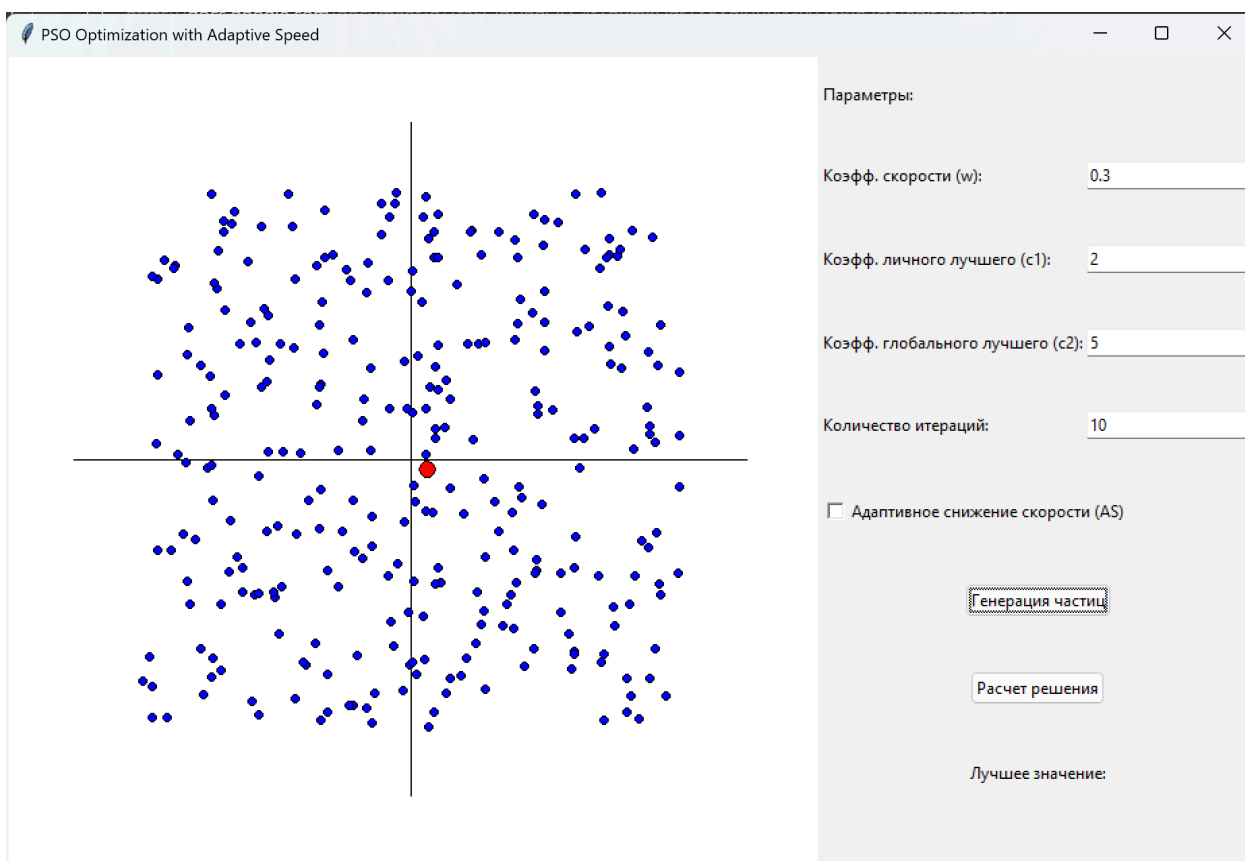


Рис 3. Результат работы алгоритма

Исследование

Тесты проводились со след параметрами: (Рис. 4)

Кэфф. скорости (w):	0.3
Кэфф. личного лучшего (c1):	2
Кэфф. глобального лучшего (c2):	5

Рис 4. Результат работы алгоритма

Таблица 2: тесты без модификации

Количество итераций	Количество частиц	Лучшее решение	Значение в лучшей точке
1	200	(1.12, -0.09)	1.08
2	200	(0.25, -0.05)	0.05
5	200	(0.13, 0.04)	0.03
10	200	(0.01, 0.01)	0.00
50	200	(0.0, -0.0)	0.00

Таблица 3: тесты с модификацией

Количество итераций	Количество частиц	Лучшее решение	Значение в лучшей точке
1	200	(-0.51, 0.28)	0.21
2	200	(-0.47, 0.08)	0.16
5	200	(0.01, -0.07)	0.01
10	200	(0.03, 0.01)	0.00
50	200	(-0.0, 0.0)	0.00

В ходе исследования были получены статистические данные, показывающие, что модификация -- ограничения скорости положительно влияет на сходимость решения. Предполагалось, что подобное улучшение позволит частицам плавно изменять свою скорость по мере приближения к минимуму функции. Это действительно видно по результатам тестов. Также можно заметить, что модификация ограничения скорости позволяет избежать

застревания в локальных минимумах и чрезмерных прыжков на начальных этапах, что опять же можно видеть по результатам тестов.

Сравнение генетического алгоритма (ГА) и роевого алгоритма (PSO)

Роевой алгоритм (PSO) и генетический алгоритм (ГА) оба показали потенциал в решении задач оптимизации, однако, они подходят к этому по-разному, и каждый имеет свои преимущества и недостатки.

Скорость сходимости и точность:

В PSO добавление модификации позволило добиться более быстрой и плавной сходимости к оптимальному значению, особенно при большом количестве итераций и частиц. Например, уже на 50-й итерации с 200 частицами алгоритм достиг значения порядка 10^{-23} , что указывает на высокую точность.

ГА, напротив, показал постепенное улучшение результатов при увеличении числа поколений и хромосом, однако достижение минимальных значений было более медленным. Например, вещественный геном с модификацией потребовал 3000 поколений для достижения значения порядка 10^{-6} .

Чувствительность к количеству частиц/хромосом:

В PSO тесты с меньшим количеством частиц (20) показали более медленную сходимость, хотя алгоритм всё же выходил на точные значения после 50 итераций, особенно при использовании модификации.

В ГА количество хромосом также играло значительную роль. Например, тесты с 50 хромосомами показали лучшие результаты, чем тесты с 20, что подтверждает, что увеличение числа хромосом способствует нахождению более качественных решений.

Эффективность модификаций:

В PSO использование модификации ограничения скорости улучшило сходимость и точность. Без модификации алгоритм показывал более медленные и менее точные результаты, особенно при малом числе частиц.

В ГА модификация также положительно сказалась на результатах.

Вывод

Выбор между PSO и ГА зависит от требований к точности и скорости сходимости. PSO, особенно с модификацией, подходит для задач, где требуется быстрое и точное решение. ГА же может быть полезен, если необходимо исследовать более широкий диапазон решений или если структура задачи требует гибкости в настройке генома и его эволюции.

Источники

1. Random documentation // RandomURL: <https://random.org> (дата обращения: 15.11.2024).
2. Tkinter documentation // Tkinter URL: <https://docs.python.org/3/library/tkinter.html> (дата обращения: 15.11.2024)